

Applied Statistics and Data Analysis

Lab 8: Tree-based methods

Luca Grassetti and Paolo Vidoni
Department of Economics and Statistics, University of Udine

December, 2022 (DRAFT)

1 Introduction

Tree-based methods for regression and classification involve stratifying or segmenting the predictor space into a number of simple non-overlapping regions. In order to make a prediction for a given observation, we typically use the mean or the mode of the training observations in the region to which it belongs. The set of splitting rules used to segment the predictor space can be summarized in a tree, typically drawn upside down. Tree-based methods are simple and useful for interpretation. However, they are usually not competitive with the best supervised learning approaches.

2 Regression trees

A simple example, with only one regressor variable, is considered. The `car.test.frame` data set has 60 rows and 8 columns, giving data on makes of cars taken from the April, 1990 issue of *Consumer Reports*.

The `car.test.frame` data set, available in the library `rpart`, contains data concerning eight variables. We consider a simple example with only one explicative variable and then we focus on:

- **Mileage**, the fuel consumption miles per US gallon, as tested (considered as the response variable);
- **Weight**, the weight of the vehicle, in pounds, including a full tank of fuel and all standard equipment (considered as the regressor).

```
str(car.test.frame)
```

```

'data.frame': 60 obs. of  8 variables:
 $ Price      : int  8895 7402 6319 6635 6599 8672 7399 7254 9599 5866 ...
 $ Country    : Factor w/ 8 levels "France","Germany",...: 8 8 5 4 3 6 4 5 3 3 ...
 $ Reliability: int   4 2 4 5 5 4 5 1 5 NA ...
 $ Mileage    : int   33 33 37 32 32 26 33 28 25 34 ...
 $ Type       : Factor w/ 6 levels "Compact","Large",...: 4 4 4 4 4 4 4 4 4 4 ...
 $ Weight     : int  2560 2345 1845 2260 2440 2285 2275 2350 2295 1900 ...
 $ Disp.      : int   97 114 81 91 113 97 97 98 109 73 ...
 $ HP         : int  113 90 63 92 103 82 90 74 90 73 ...

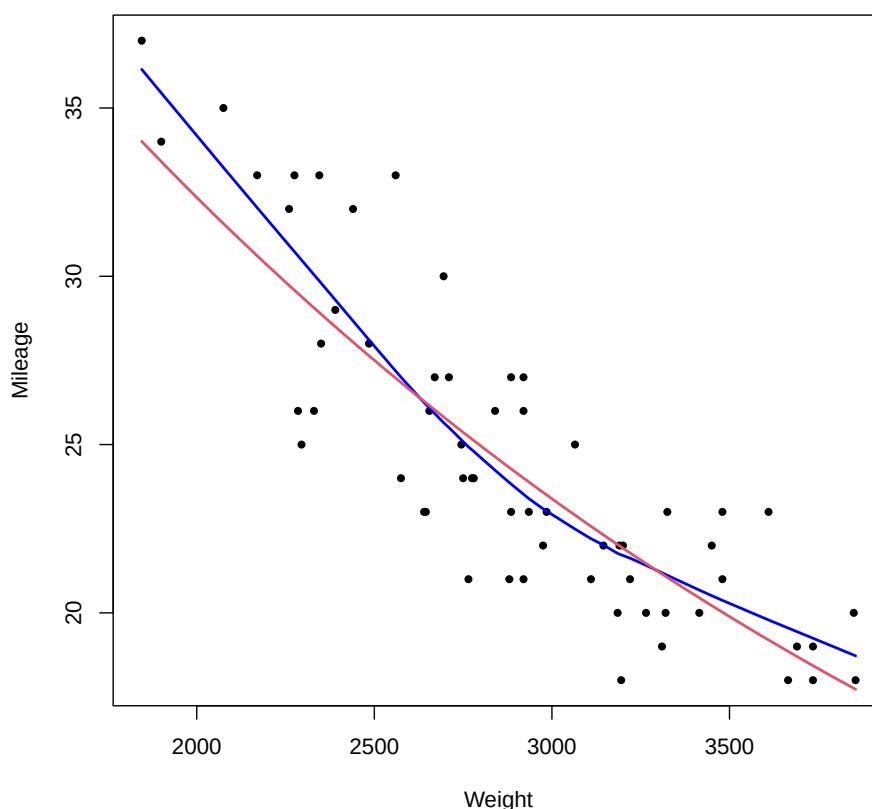
```

The scatterplot with the fitted smooth curve in blue and the fitted regression curve in red (based on a simple linear regression for the log response) is given below.

```

plot(Mileage~Weight, data = car.test.frame, xlab = 'Weight', ylab = 'Mileage',
     cex=0.8, pch = 16)
with(car.test.frame, lines(lowess(Mileage~Weight, f=.7), lwd=2, col='blue'))
mod.lm <- lm(log(Mileage)~Weight,car.test.frame)
wseq <- seq(min(car.test.frame$Weight), max(car.test.frame$Weight), l=30)
lines(wseq, exp(predict(mod.lm, data.frame(Weight=wseq))),col=2, lwd=2)

```



In order to define a regression tree, the function `rpart` of the library `rpart` is considered. This function builds regression and classification trees using a suitable recursive partitioning approach. The main arguments are:

- `formula`, which is an object giving the symbolic description of the model, with a response, the regressors but no interaction terms;
- `data`, which gives the data frame, if required;
- `method`, which specifies the objective function used in the splitting procedure; in particular, "anova" is used for numerical response variables and it corresponds to the residual sum of squares and "class" is assumed for a categorical response; if `method` is missing then the routine tries to make an intelligent guess; there is also the possibility of considering user written split functions;
- `parms`, which gives optional parameters for the splitting function: the "anova" splitting has no parameters, for the classification splitting ("class"), we may consider the Gini index (corresponding to the default choice) `parms=list(split="gini")` or the entropy based index `parms=list(split="information")`;
- `control = rpart.control()`, which is a list of possible options that control details of the splitting algorithm; in particular, `minbucket` provides the smallest number of observations that are allowed in a terminal node, `minsplit` sets the smallest number of observations in the parent node that could be split further and `cp` defines the value of the complexity parameter (the default value is 0.01).

The `rpart` function defines an object of class `rpart`. This output can be used to draw the tree by considering the function `plot`, where the text describing the nodes is included using the `text` function. Moreover, the function `summary` provides the CP Table, the measures of variable importance, the description of the nodes and the splits. In this case, we obtain a very simple regression tree, where the splits involve the single regressor `Weight`.

```
car.tree <- rpart(Mileage ~ Weight, data=car.test.frame, method="anova")
summary(car.tree)
```

Call:

```
rpart(formula = Mileage ~ Weight, data = car.test.frame, method = "anova")
n= 60
```

	CP	nsplit	rel error	xerror	xstd
1	0.59534912	0	1.0000000	1.0235073	0.17503378
2	0.13452819	1	0.4046509	0.5406360	0.10539052
3	0.01282843	2	0.2701227	0.3964894	0.07896010
4	0.01000000	3	0.2572943	0.3919595	0.07111917

Variable importance
Weight

100

```
Node number 1: 60 observations,    complexity param=0.5953491
  mean=24.58333, MSE=22.57639
  left son=2 (45 obs) right son=3 (15 obs)
  Primary splits:
    Weight < 2567.5 to the right, improve=0.5953491, (0 missing)

Node number 2: 45 observations,    complexity param=0.1345282
  mean=22.46667, MSE=8.026667
  left son=4 (22 obs) right son=5 (23 obs)
  Primary splits:
    Weight < 3087.5 to the right, improve=0.5045118, (0 missing)

Node number 3: 15 observations
  mean=30.93333, MSE=12.46222

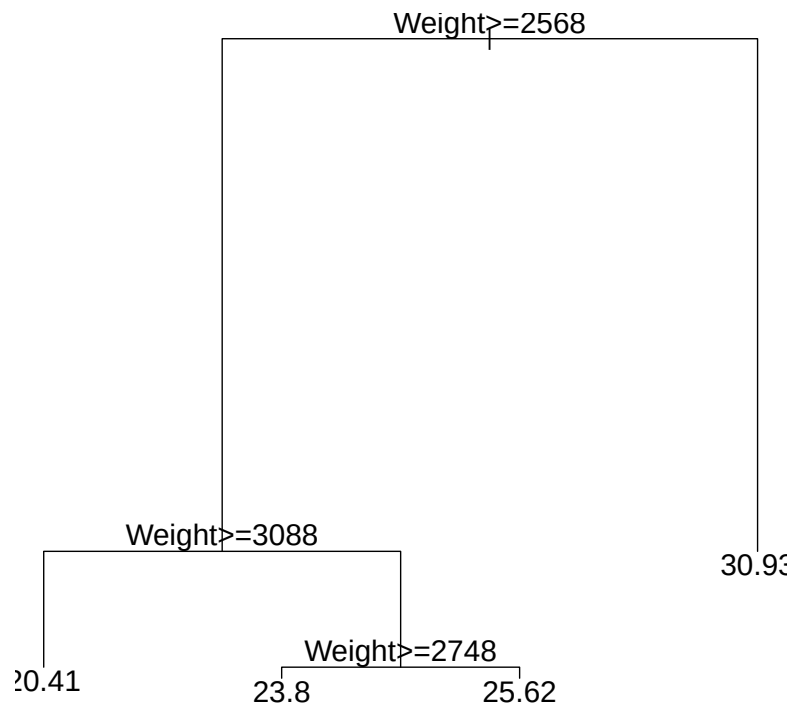
Node number 4: 22 observations
  mean=20.40909, MSE=2.78719

Node number 5: 23 observations,    complexity param=0.01282843
  mean=24.43478, MSE=5.115312
  left son=10 (15 obs) right son=11 (8 obs)
  Primary splits:
    Weight < 2747.5 to the right, improve=0.1476996, (0 missing)

Node number 10: 15 observations
  mean=23.8, MSE=4.026667

Node number 11: 8 observations
  mean=25.625, MSE=4.984375

plot(car.tree)
text(car.tree, cex=1.3)
```



Furthermore, with the `predict()` command, it is possible to apply the model in order to get predictions; with the option `newdata` it is possible to consider a new set of data. The prediction is defined as the mean of the training observations in the region to which the new data belongs, if we set `method="anova"`. In case of a categorical response variable, using the `"class"` method, it will give the most likely class label. In this last situation, it is possible to define the type of predicted value to be returned: the default (`type="prob"`) is a matrix of class probabilities and, with `type="vector"`, we get a vector of predicted responses (for regression trees this is the mean response at the terminal node and for classification trees it is the predicted class as a number), with `type="class"` (for a classification tree) a factor of classifications based on the responses and, finally, with `type="matrix"`, a matrix of the full responses.

```
predict(car.tree, data.frame(Weight=c(2780)), method="anova")
```

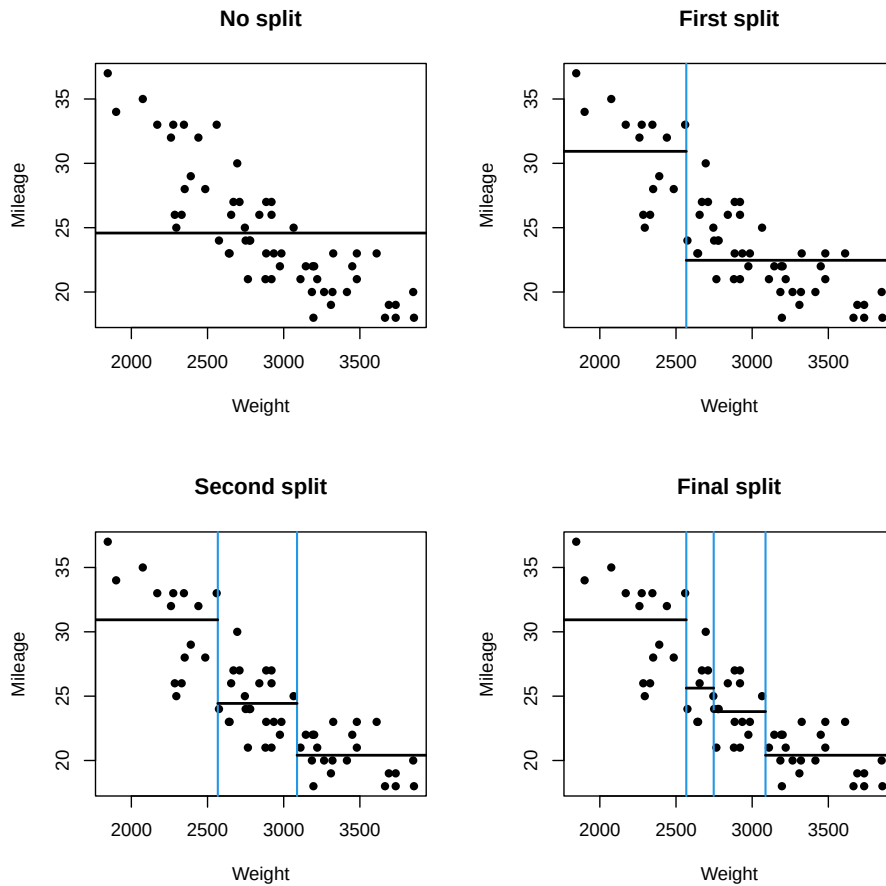
```
1
23.8
```

In the following commands, we describe the sequence of partitions of the one-dimensional regression space (blue line) by recursive binary splitting and the associated point predictors for each box (black line). The sequence of splits is obtained by setting the tuning parameter `cp` at a suitable level.

```

car1 <- rpart(Mileage ~ Weight, data=car.test.frame, method="anova",
control=list(cp=0.5))
y1 <- predict(car1)
car2 <- rpart(Mileage ~ Weight, data=car.test.frame, method="anova",
control=list(cp=0.1))
y2 <- predict(car2)
y3 <- predict(car.tree)
par(mfrow=c(2,2))
with(car.test.frame, plot(Mileage~Weight, pch=16))
abline(h=mean(car.test.frame$Mileage), col=1, lwd=2)
title("No split")
with(car.test.frame, plot(Mileage~Weight, pch=16))
abline(v=2567.5, col=4, lwd=1.5)
segments(1500, max(y1), 2567.5, max(y1), col=1, lwd=2)
segments(2567.5, min(y1), 4000, min(y1), col=1, lwd=2)
title("First split")
with(car.test.frame, plot(Mileage~Weight, pch=16))
abline(v=2567.5, col=4, lwd=1.5)
abline(v=3087.5, col=4, lwd=1.5)
#points(car.test.frame$Weight, y2, col=6)
segments(1500, max(y2), 2567.5, max(y2), col=1, lwd=2)
segments(2567.5, 24.43478, 3087.5, 24.43478, col=1, lwd=2)
segments(3087.5, min(y2), 4000, min(y2), col=1, lwd=2)
title("Second split")
with(car.test.frame, plot(Mileage~Weight, pch=16))
abline(v=2567.5, col=4, lwd=1.5)
abline(v=3087.5, col=4, lwd=1.5)
abline(v=2747.5, col=4, lwd=1.5)
#points(car.test.frame$Weight, y3, col=6)
segments(1500, max(y3), 2567.5, max(y3), col=1, lwd=2)
segments(2567.5, 25.625, 2747.5, 25.625, col=1, lwd=2)
segments(2747.5, 23.8, 3087.5, 23.8, col=1, lwd=2)
segments(3087.5, min(y3), 4000, min(y3), col=1, lwd=2)
title("Final split")

```



A second example concerns the data set `Hitters` of the library `ISLR`, containing $n = 322$ observations on twenty variables regarding major league baseball players from the 1986 and 1987 seasons. The response variable `Salary` is the 1987 annual salary in thousands of dollars and the following regressors are taken into account:

- `Years`: number of years in the major league;
- `Hits`: number of hits in 1986;
- `RBI`: number of runs batted in 1986;
- `PutOuts`: number of put outs in 1986;
- `Walks`: number of walks in 1986;
- `Runs`: number of runs batted in in 1986.

```
str(Hitters)

'data.frame': 322 obs. of 20 variables:
 $ AtBat    : int  293 315 479 496 321 594 185 298 323 401 ...
```

```

$ Hits      : int  66 81 130 141 87 169 37 73 81 92 ...
$ HmRun     : int   1 7 18 20 10 4 1 0 6 17 ...
$ Runs      : int  30 24 66 65 39 74 23 24 26 49 ...
$ RBI       : int  29 38 72 78 42 51 8 24 32 66 ...
$ Walks     : int  14 39 76 37 30 35 21 7 8 65 ...
$ Years     : int   1 14 3 11 2 11 2 3 2 13 ...
$ CAtBat    : int 293 3449 1624 5628 396 4408 214 509 341 5206 ...
$ CHits     : int  66 835 457 1575 101 1133 42 108 86 1332 ...
$ CHmRun    : int   1 69 63 225 12 19 1 0 6 253 ...
$ CRuns     : int  30 321 224 828 48 501 30 41 32 784 ...
$ CRBI      : int  29 414 266 838 46 336 9 37 34 890 ...
$ CWalks    : int  14 375 263 354 33 194 24 12 8 866 ...
$ League    : Factor w/ 2 levels "A","N": 1 2 1 2 2 1 2 1 2 1 ...
$ Division  : Factor w/ 2 levels "E","W": 1 2 2 1 1 2 1 2 2 1 ...
$ PutOuts   : int  446 632 880 200 805 282 76 121 143 0 ...
$ Assists   : int   33 43 82 11 40 421 127 283 290 0 ...
$ Errors    : int   20 10 14 3 4 25 7 9 19 0 ...
$ Salary    : num   NA 475 480 500 91.5 750 70 100 75 1100 ...
$ NewLeague : Factor w/ 2 levels "A","N": 1 2 1 2 2 1 1 1 2 1 ...

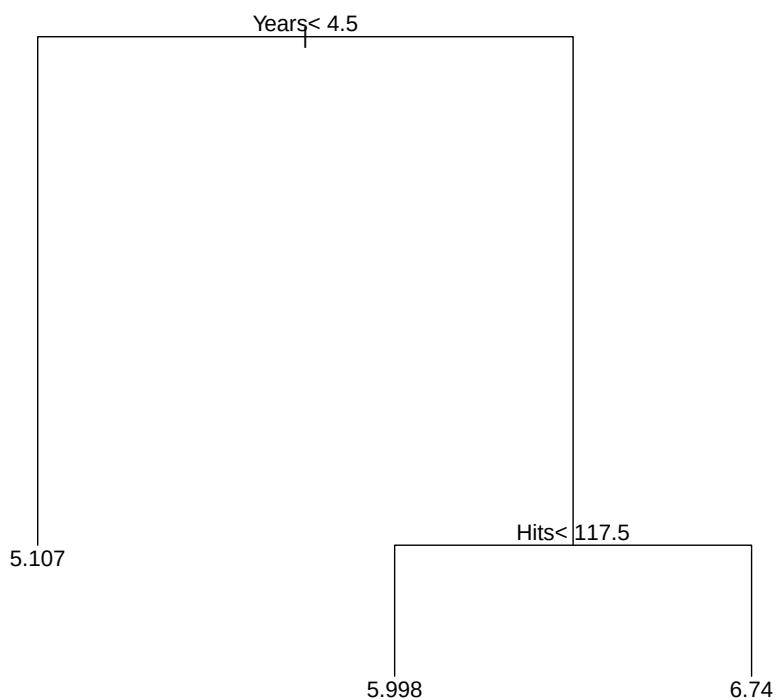
```

We consider initially only the regressors **Hits** and **Years** and the aim is to predict a baseball player's **Salary** based on these two explanatory variables. We consider the log-transform **Salary** so that its distribution has more of a typical bell-shape. Using the **rpart** function, we obtain the following regression tree, assuming the value 0.1 for the tuning parameter **cp** and excluding, with the argument **subset = complete.cases(Hitters)**, the training observations with NA values.

```

hitter.tree <- rpart(log(Salary) ~ Hits+Years, data=Hitters, method="anova",
                     subset = complete.cases(Hitters), control=list(cp=0.1))
plot(hitter.tree)
text(hitter.tree,cex=1)

```

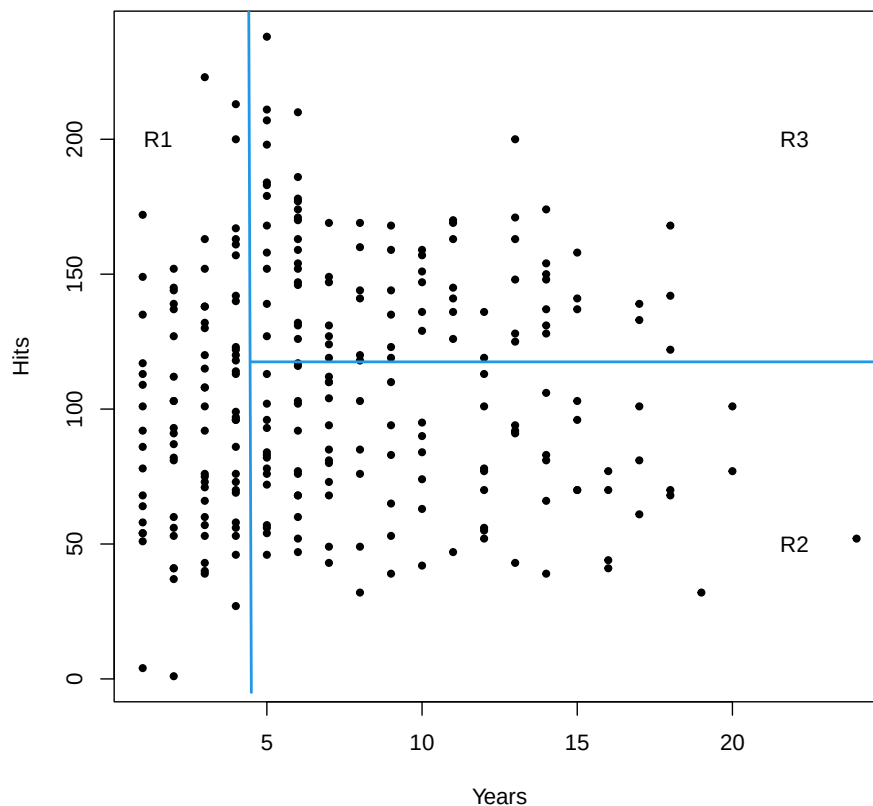


The three-region partition for the `Hitters` data set, from this regression tree, is illustrated in the following figure.

```

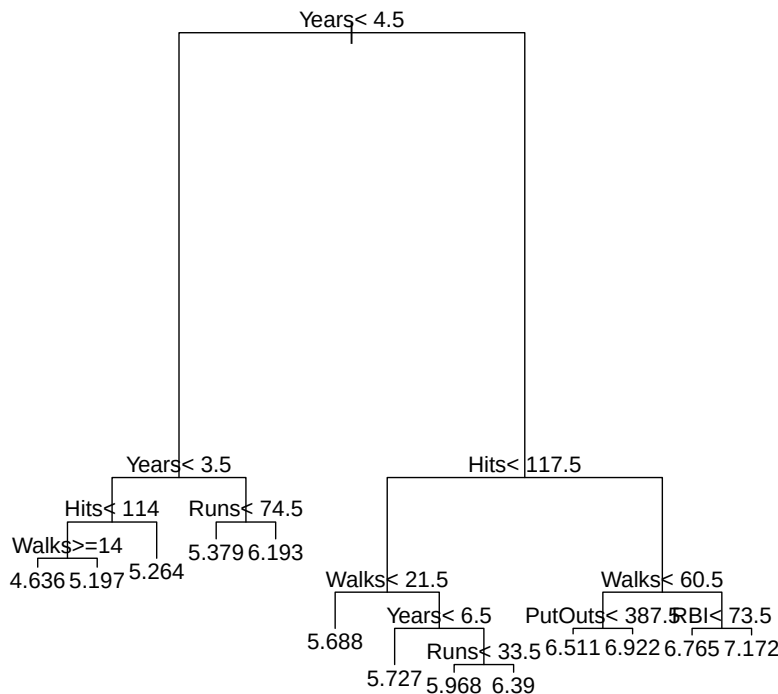
plot(Hits ~ Years, data=Hitters, subset = complete.cases(Hitters), pch=20)
segments(4.5, -5, 4.4, 300, col=4, lwd=2)
segments(4.5, 117.5, 30, 117.5, col=4, lwd=2)
text(1.5, 200, labels = 'R1', cex=1)
text(22, 50, labels = 'R2', cex=1)
text(22, 200, labels = 'R3', cex=1)

```



The result of fitting a regression tree on the `Hitters` data, using six regressor and setting `cp=0.005`, is represented below. We fix the seed for random number generation, since the `rpart` function produces also the cross-validation estimates of the test mean square error, with alternative values for the tuning parameter `cp`.

```
set.seed(11)
hitter.tree1 <- rpart(log(Salary) ~ Hits+Years+RBI+PutOuts+Walks+Runs, data=Hitters,
                      method="anova", subset = complete.cases(Hitters), control=list(cp=0.005))
plot(hitter.tree1)
text(hitter.tree1, cex=1)
```



This regression tree may produce good predictions on the training set, but is likely to overfit the data, leading to poor test set performance, since it might be too complex. It is possible to prune the tree in order to obtain a subtree having a lower test error rate. Given a subtree, we can estimate its test error using cross-validation, however, estimating the cross-validation error for every possible subtree would be too cumbersome, since there is an extremely large number of possible subtrees. Instead, we need a way to select a small set of subtrees for consideration. A possible approach is based on *cost complexity pruning* and, instead of considering every possible subtree, we consider a sequence of trees indexed by a non-negative tuning parameter. A subtree is selected by considering that one with the minimum cross-validated error rate or that one satisfying the *one standard deviation rule*.

The `cptable` included in the `rpart` object `hitter.tree1` is useful for cost complexity pruning. With regard to a sequence of values for the tuning parameter `cp`, it provides the training mean square error (third column), the CV estimate of the test mean square error (forth column) and the associated estimated standard error (last column). The second column gives the number of splits for each tree (the number of leaves, that is terminal nodes, is obtained by adding one). In the following plot, obtained using the function `plotcp`, the CV estimate of the test errors, with the associated standard error bars, are represented in black as function of `cp` and of the size of the tree. The training error curve is shown in red. The horizontal blue line is drawn at one standard deviation above the minimum of CV error.

```
hitter.tree1$variable.importance
```

Years	Hits	Runs	RBI	Walks	PutOuts
107.15400	37.80858	35.14709	29.25716	28.85977	12.98755

```
hitter.tree1$cptable
```

	CP	nsplit	rel error	xerror	xstd
1	0.444574455	0	1.0000000	1.0112651	0.06552817
2	0.114545498	1	0.5554255	0.5640635	0.05889020
3	0.044460214	2	0.4408800	0.4632533	0.05742060
4	0.018312680	3	0.3964198	0.4547599	0.06209986
5	0.018066887	4	0.3781072	0.4993874	0.06707068
6	0.016786324	6	0.3419734	0.4919949	0.06715229
7	0.016177383	7	0.3251871	0.4989911	0.07094600
8	0.008893003	8	0.3090097	0.4539000	0.06828076
9	0.007658643	9	0.3001167	0.4737207	0.07000742
10	0.006566393	10	0.2924580	0.4721652	0.07113095
11	0.005494065	11	0.2858916	0.4716711	0.07119854
12	0.005000000	12	0.2803976	0.4800862	0.07566399

```
plotcp(hitter.tree1,lwd=2)
```

```
lines(1:12,hitter.tree1$cptable[,3],col=2,type="o",lwd=2)
```

```
cverror<-hitter.tree1$cptable[,4]
```

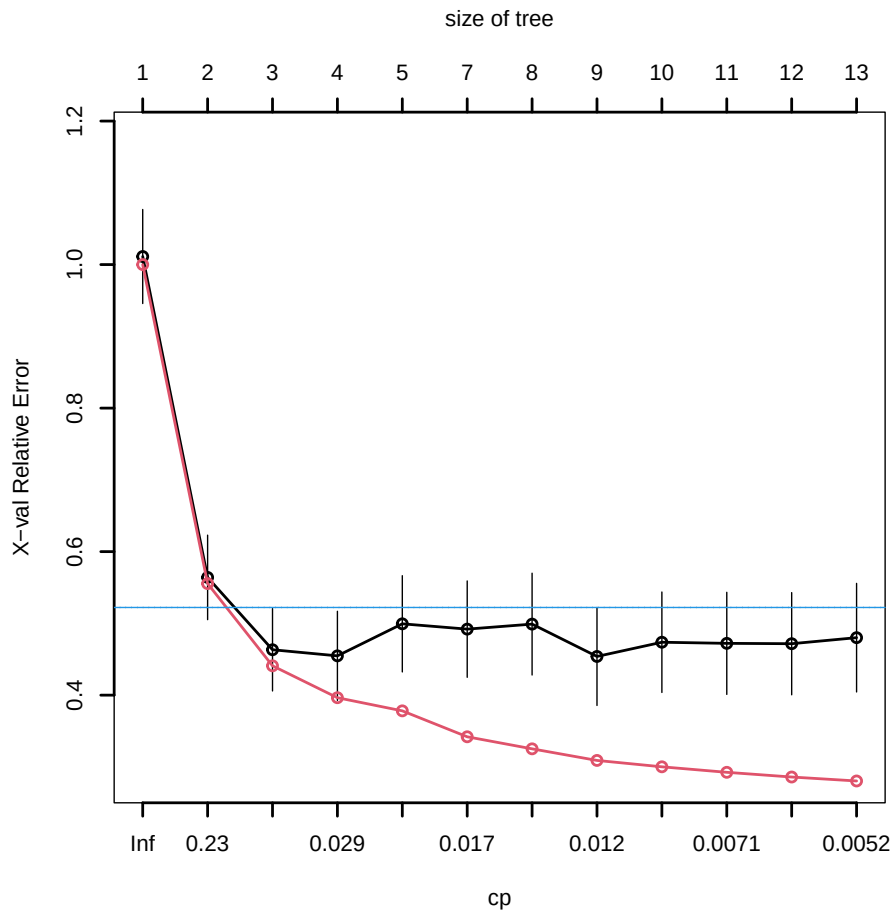
```
minxe <- which.min(cverror) # subtree with the lowest cverror
```

```
minxe
```

```
8
```

```
8
```

```
abline(h=hitter.tree1$cptable[minxe, 4] + hitter.tree1$cptable[minxe, 5],col=4)
```



```
ind <- which.max(cverror <= hitter.tree1$cptable[minxe, 4] +
  hitter.tree1$cptable[minxe, 5])
# subtree satisfying the one standard deviation rule
ind

3
3

cpind <- hitter.tree1$cptable[ind,1]
cpind # cp for the subtree satisfying the one standard deviation rule

[1] 0.04446021

cpopt <- hitter.tree1$cptable[minxe,1]
cpopt # cp for the subtree with the lowest cverror

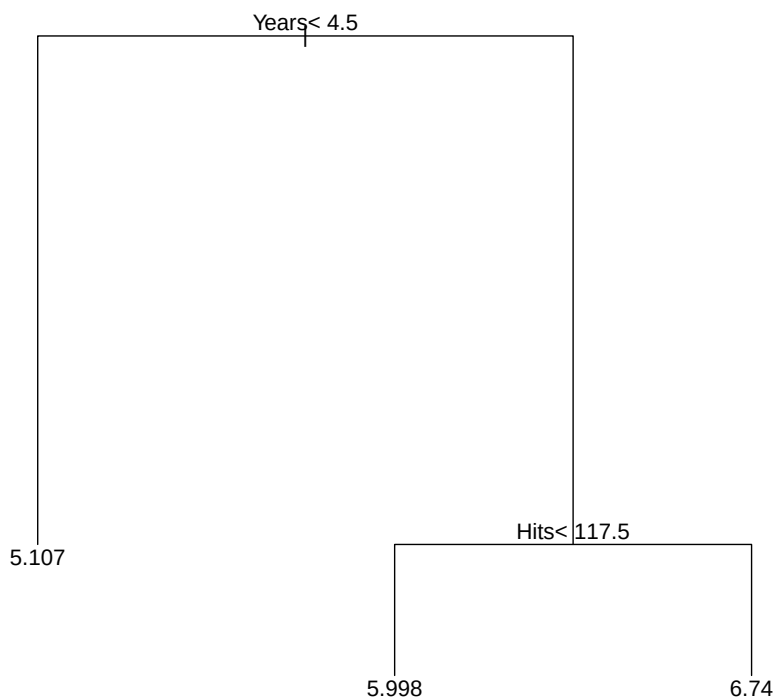
[1] 0.008893003
```

The minimum CV error occurs at a tree of size 4, whereas the *one standard deviation rule* selects the tree of size 3, represented below.

```

hitter.tree2 <- rpart(log(Salary) ~ Hits+Years+RBI+PutOuts+Walks+Runs, data=Hitters,
                      method="anova", subset = complete.cases(Hitters), control=list(cp=cpind))
plot(hitter.tree2)
text(hitter.tree2, cex=1)

```



3 Classification trees

The `spam7` data set of the library `DAAG` contains data on 4601 email items, of which 1813 were identified as spam. In the original data set there are 57 explanatory variables; a reduced version with the following 6 explanatory variables is taken into account:

- `crl.tot`: total length of words in capitals;
- `dollar`: number of occurrences of the \$ symbol;
- `bang`: number of occurrences of the ! symbol;
- `n000`: number of occurrences of the string 000;
- `make`: number of occurrences of the word make;

- **money**: number of occurrences of the word money.

The response variable is **yesno**, which is a factor with two levels: **n** not spam, **y** spam.

```
str(spam7)

'data.frame': 4601 obs. of 7 variables:
 $ crl.tot: num 278 1028 2259 191 191 ...
 $ dollar : num 0 0.18 0.184 0 0 0 0.054 0 0.203 0.081 ...
 $ bang : num 0.778 0.372 0.276 0.137 0.135 0 0.164 0 0.181 0.244 ...
 $ money : num 0 0.43 0.06 0 0 0 0 0 0.15 0 ...
 $ n000 : num 0 0.43 1.16 0 0 0 0 0 0 0.19 ...
 $ make : num 0 0.21 0.06 0 0 0 0 0 0.15 0.06 ...
 $ yesno : Factor w/ 2 levels "n","y": 2 2 2 2 2 2 2 2 2 2 ...
```

We use the same commands as before. The only difference is that, with the argument **method="class"** considered in the function **rpart**, we apply the splitting procedure by considering the Gini index. However, in the cost complexity pruning procedure the CV estimate of the usual classification error is considered.

```
set.seed(11)
spam.rpart <- rpart(yesno~crl.tot+dollar+bang+money+n000+make, method="class",
                    data=spam7, cp=0.0005)
spam.rpart$variable.importance
```

dollar	bang	money	n000	crl.tot	make
771.85435	440.59881	341.95847	312.93283	277.34258	46.07674

```
printcp(spam.rpart)
```

Classification tree:

```
rpart(formula = yesno ~ crl.tot + dollar + bang + money + n000 +
      make, data = spam7, method = "class", cp = 5e-04)
```

Variables actually used in tree construction:

```
[1] bang    crl.tot  dollar  make    money    n000
```

Root node error: 1813/4601 = 0.39404

n= 4601

	CP	nsplit	rel error	xerror	xstd
1	0.47655819	0	1.00000	1.00000	0.018282
2	0.07556536	1	0.52344	0.54164	0.015329

3	0.01158301	3	0.37231	0.38720	0.013453
4	0.01047987	4	0.36073	0.39051	0.013500
5	0.00634308	5	0.35025	0.37397	0.013262
6	0.00551572	10	0.31660	0.35852	0.013031
7	0.00441258	11	0.31109	0.34528	0.012827
8	0.00386100	12	0.30667	0.33701	0.012696
9	0.00275786	16	0.29123	0.32763	0.012545
10	0.00220629	17	0.28847	0.32212	0.012455
11	0.00193050	18	0.28627	0.32212	0.012455
12	0.00165472	20	0.28240	0.32322	0.012473
13	0.00091929	25	0.27413	0.32157	0.012446
14	0.00082736	29	0.26917	0.32212	0.012455
15	0.00055157	46	0.25427	0.32267	0.012464
16	0.00050000	48	0.25317	0.33149	0.012608

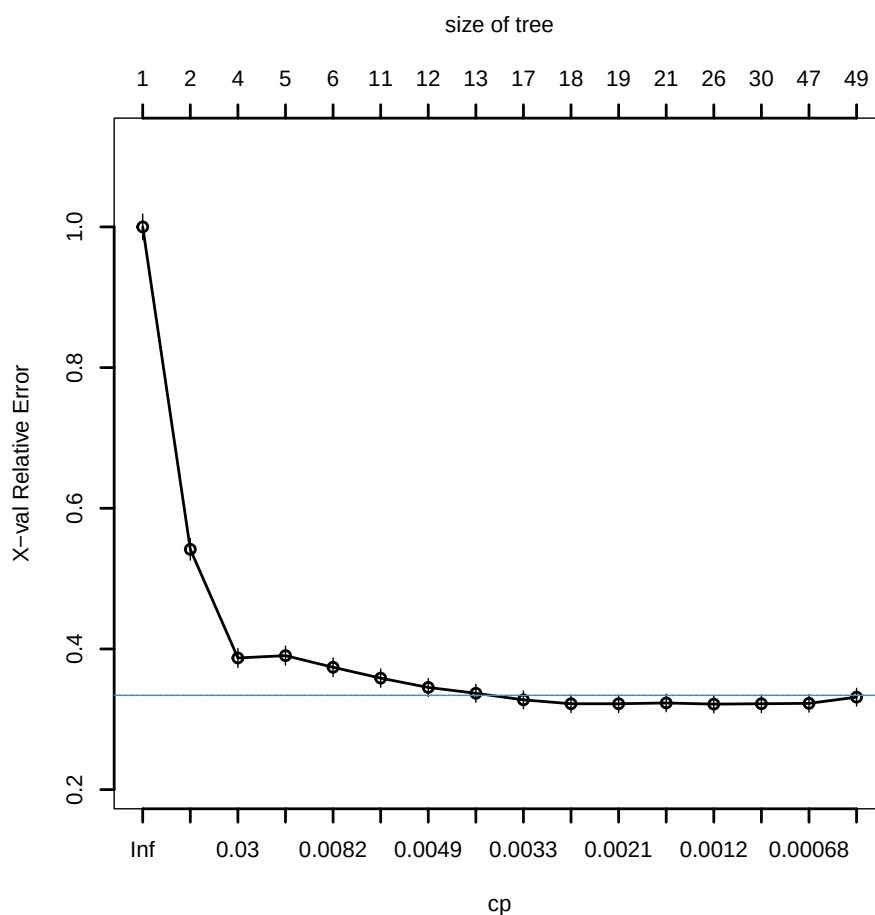
```

plotcp(spam.rpart,lwd=2)
tab <- spam.rpart$scptable
cverror<-tab[,4]
minxe <- which.min(cverror) # subtree with the lowest cverror
minxe

13
13

abline(h=tab[minxe, 4] + tab[minxe, 5],col=4)

```



```
ind <- which.max(cverror <= tab[minxe, 4] + tab[minxe, 5])
# subtree satisfying the one standard deviation rule
ind

9
9

# cp for the subtree satisfying the one standard deviation rule
cpind <- tab[ind,1]
cpind

[1] 0.00275786

# cp for the subtree with the lowest cverror
cpopt <- tab [minxe,1]
cpopt

[1] 0.0009192866
```

With regard to the spam data set, a large tree is fitted to the training data by setting `cp=0.0005`. The resulting classifier is too complex, having 49 leaves. A cost complexity pruning procedure is applied. The above plot describes the CV classification error, corresponding to different values of `cp`. The horizontal blue line is drawn at one standard deviation above the minimum of CV error. The minimum CV error occurs at a tree of size 18, whereas the one standard deviation rule selects a tree of size 17. With this latter choice, `cp = 0.0028` and the estimate of the test classification error is 12.9%.

```

rooter <- min(as.vector(table(spam.rpart$y)/nrow(spam7)))
# error rate of the constant classifier
# rooter <- min(as.vector(table(spam7$yesno)/nrow(spam7)))
rooter

[1] 0.3940448

# Prediction error rate in training data = error rate of the constant
# classifier * rel error * 100%
# Prediction error rate in cross-validation = error rate of the constant
# classifier * xerror * 100%
rounder <- round(100 * tab[ind,4] * rooter,1)
rounder

[1] 12.9

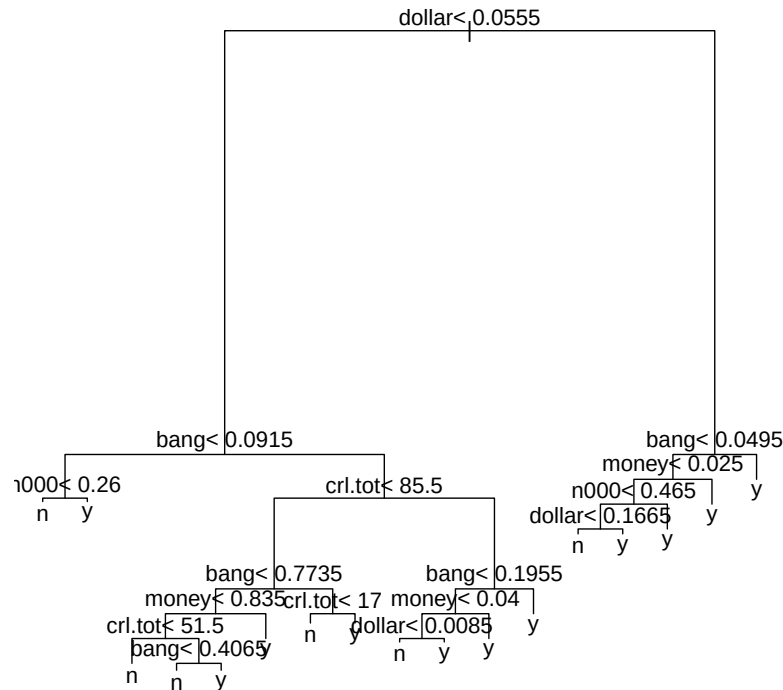
```

The final classifier is complex, but interpretable.

```

spam.rpartcp <- rpart(yesno~crl.tot+dollar+bang+money+n000+make, method="class",
                      data=spam7, cp=cpind)
plot(spam.rpartcp)
text(spam.rpartcp,cex=1)

```



4 Ensemble methods

The function `randomForest` of the package `randomForest` implements the Breiman's random forest algorithm (and also the Bagging procedure) for classification and regression. The main arguments are:

- **formula**: a formula object describing the model to be fitted (as an alternative, the data frame or the matrix `x` of predictors and the response variable observations `y`);
- **data**: the data set, if required;
- **ntree**: the number of trees to grow (the default is 500);
- **mtry**: the number of variables randomly sampled as candidates at each split (if this argument is set equal to the number of regressors we have the bagging procedure);
- **importance**: if `TRUE`, the importance of the predictors is assessed.

The `plot` function, applied to a `randomForest` type object, draws the error rate or the MSE as a function of the number of bootstrap samples considered in the procedure.

The first applications concerns the `Hitters` data set.

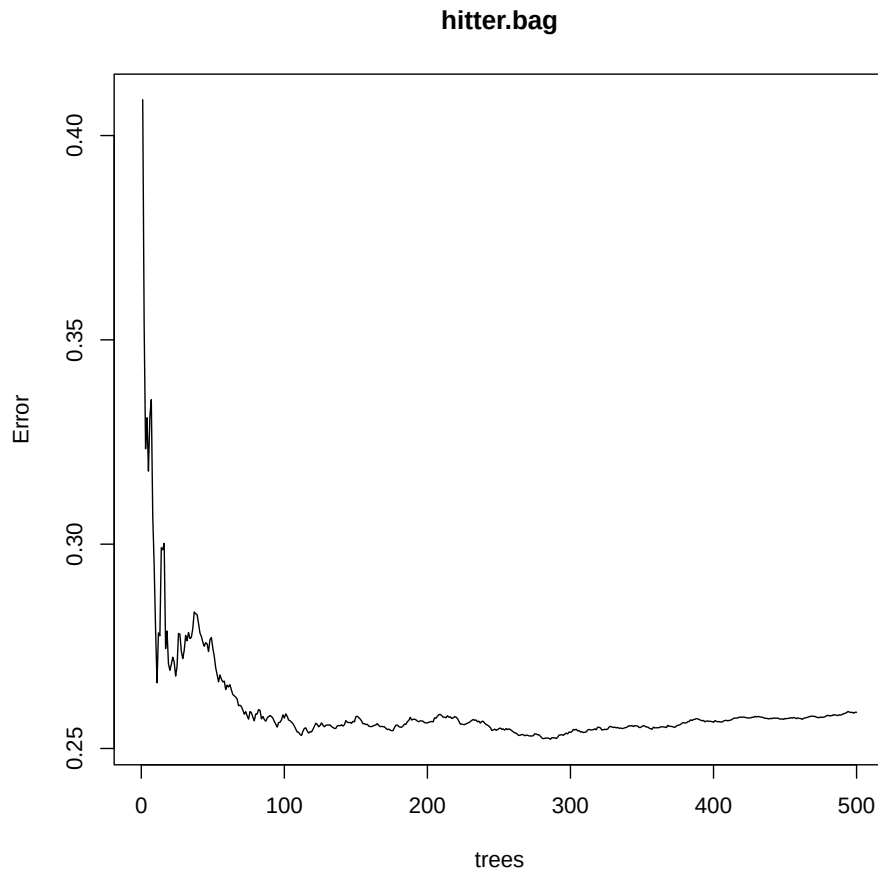
We consider at first the bagging solution and then a random forest with 4 as the number of predictors sampled for splitting.

```
Hitters.compl <- Hitters[complete.cases(Hitters),] # dataframe with complete cases
# bagging
set.seed(11)
hitter.bag <- randomForest(log(Salary) ~ Hits+Years+RBI+PutOuts+Walks+Runs,
                           data=Hitters.compl, mtry=6, importance=TRUE)
hitter.bag

Call:
randomForest(formula = log(Salary) ~ Hits + Years + RBI + PutOuts + Walks + Runs, data = Hitters.compl,
              mtry = 6, importance = TRUE)
Type of random forest: regression
Number of trees: 500
No. of variables tried at each split: 6

Mean of squared residuals: 0.2588569
% Var explained: 67.14

plot(hitter.bag)
```



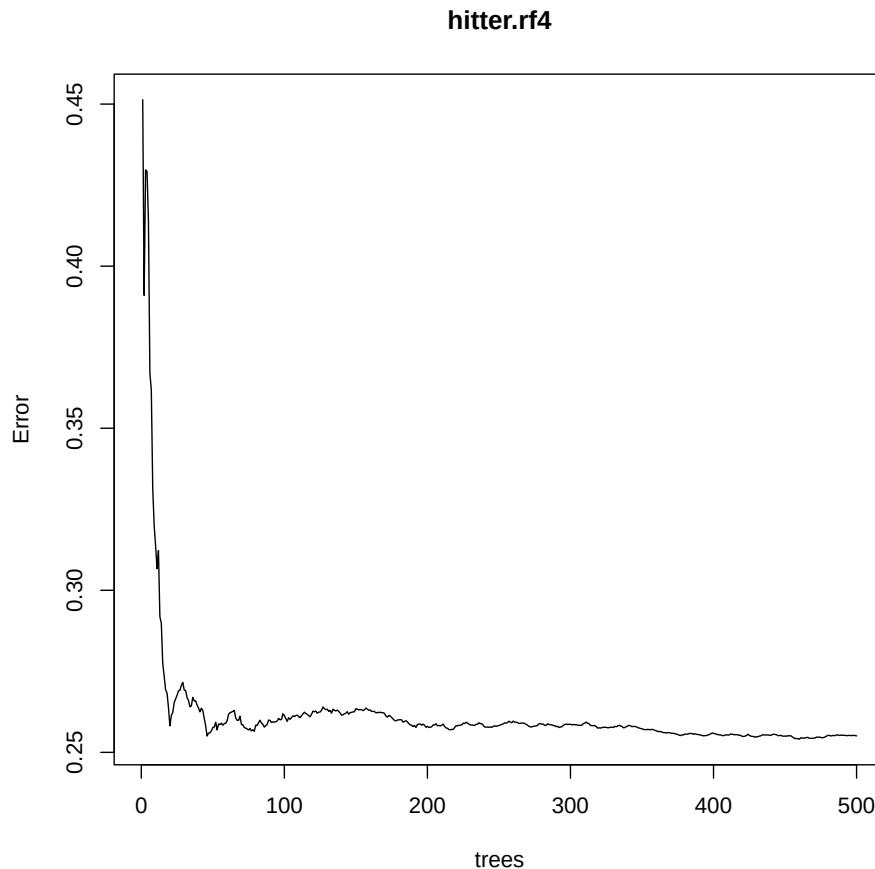
```
# random forest with m=4
set.seed(11)
hitter.rf4 <- randomForest(log(Salary) ~ Hits+Years+RBI+PutOuts+Walks+Runs,
                           data=Hitters.compl, mtry=4, importance=TRUE)
hitter.rf4
```

Call:

```
randomForest(formula = log(Salary) ~ Hits + Years + RBI + PutOuts + Walks + Runs, data = Hitters.compl,
              mtry = 4, importance = TRUE)
Type of random forest: regression
Number of trees: 500
No. of variables tried at each split: 4
```

```
Mean of squared residuals: 0.2550826
% Var explained: 67.62
```

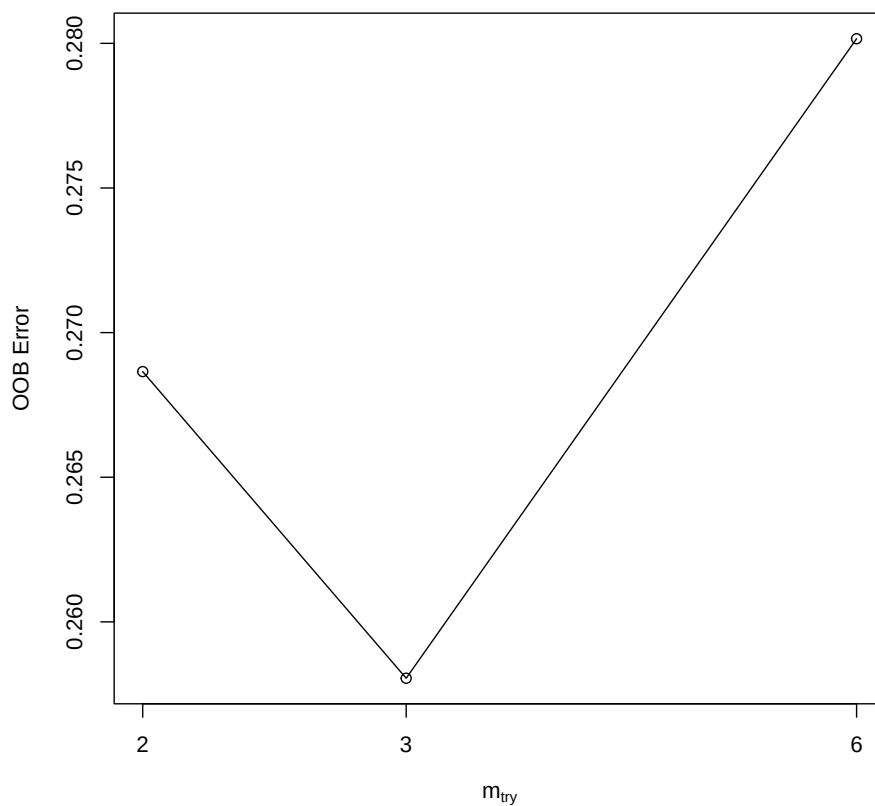
```
plot(hitter.rf4)
```



Function `tuneRF`, starting with a default value of `mtry` set by the argument `mtryStart`, searches for the optimal value (with respect to Out-of-Bag error estimate) of `mtry` for `randomForest`. The main arguments are the matrix or data frame `x` of predictor variables and the vector `y` of response variable observations. In this case, the optimal value is 3.

```
# optimal choice of m using the OOB procedure
set.seed(1)
training.resp<-log(Hitters.compl$Salary)
training.regr<-subset(Hitters.compl, select= c(Hits,Years,RBI,PutOuts,Walks,Runs))
tuneRF(training.regr, training.resp, mtryStart=6)

mtry = 6  OOB error = 0.28016
Searching left ...
mtry = 3  OOB error = 0.258052
0.07891219 0.05
mtry = 2  OOB error = 0.2686522
-0.04107795 0.05
Searching right ...
```



```

mtry  OOBError
2     2 0.2686522
3     3 0.2580520
6     6 0.2801600

```

```

# random forest with m=3
set.seed(11)
hitter.rf3 <- randomForest(log(Salary) ~ Hits+Years+RBI+PutOuts+Walks+Runs,
                           data=Hitters.compl, mtry=3, importance=TRUE)
hitter.rf3

```

Call:

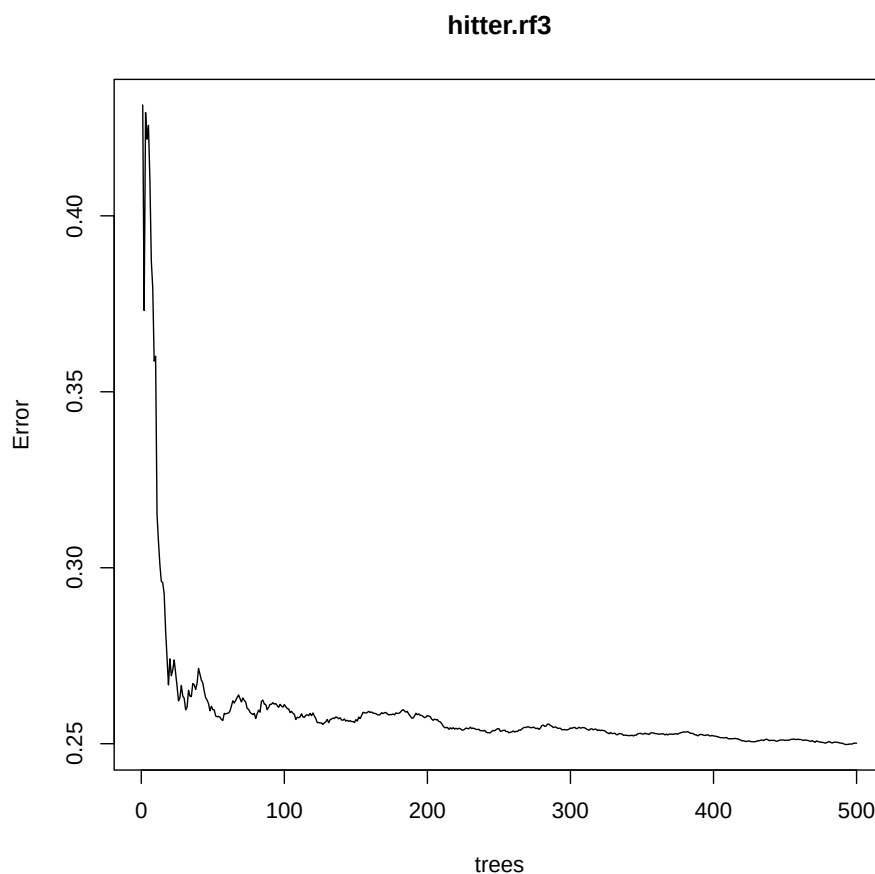
```

randomForest(formula = log(Salary) ~ Hits + Years + RBI + PutOuts + Walks + Runs, data
              Type of random forest: regression
              Number of trees: 500
No. of variables tried at each split: 3

```

```
Mean of squared residuals: 0.2501389
% Var explained: 68.24
```

```
plot(hitter.rf3)
```



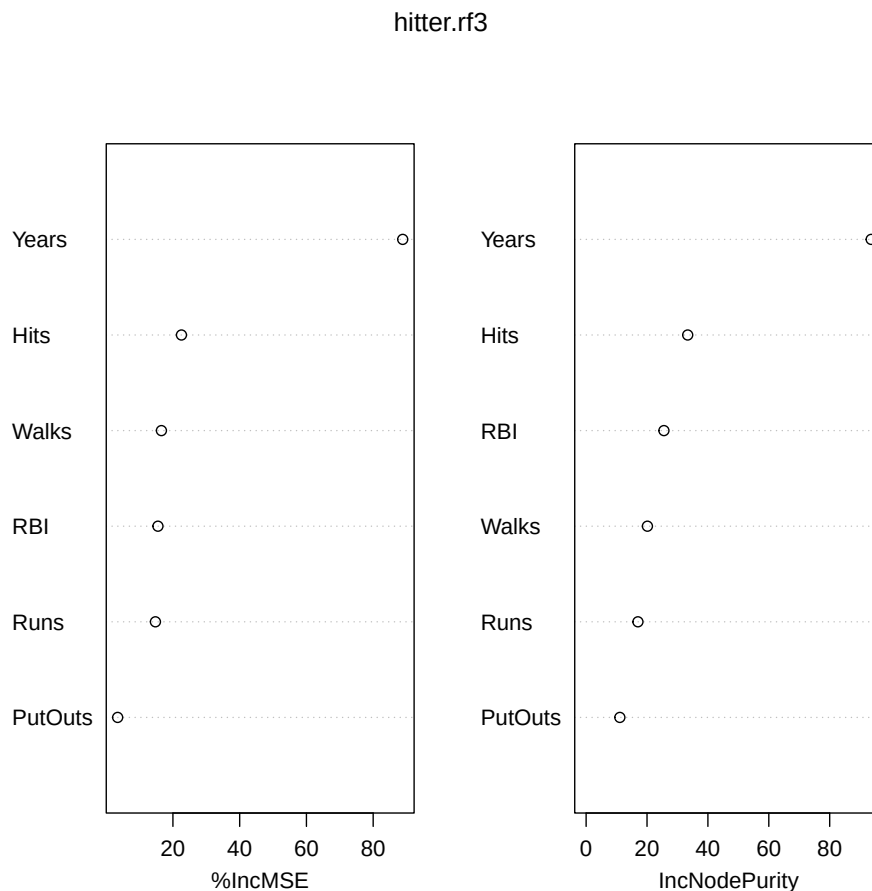
It is possible to evaluate the variable importance, and to draw the associated dotchart. To this end, we consider the output of function `importance`, which is a matrix with `nclass + 2` (for classification) or two (for regression) columns. For classification, the first `nclass` columns are the class-specific measures computed as mean decrease in accuracy. The `nclass + 1st` column is the mean decrease in accuracy over all classes. The last column is the mean decrease in Gini index. For regression, the first column is the mean decrease in accuracy and the second the mean decrease in MSE.

```
# regressor variable importance measures
importance(hitter.rf3)
```

	%IncMSE	IncNodePurity
Hits	22.55776	33.35929
Years	88.84599	93.62382
RBI	15.51350	25.55354

PutOuts	3.44298	11.07595
Walks	16.55374	20.13189
Runs	14.75764	17.01371

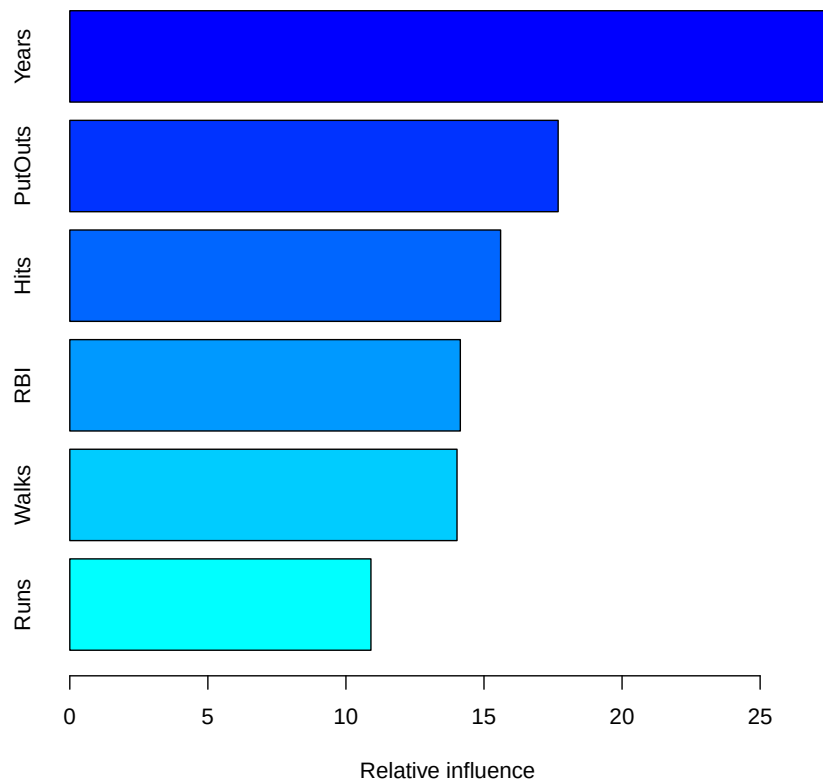
```
varImpPlot(hitter.rf3)
```



The function `gbm` of the package `gbm` fits generalized boosted regression models, for both regression and classification problems. The main arguments are the `formula` object specifying the model, `distribution` which sets the distribution to be used (if the response has only 2 unique values, `bernoulli` is assumed and, in case of a numerical response, one possibility is `gaussian`, which is related to the MSE). Moreover, `n.trees` defines the total number of trees to fit (this is equivalent to the number of iterations), with default value 100, `interaction.depth` is an integer specifying the maximum depth of each tree (the default is 1) and `shrinkage` is the learning rate parameter applied to each tree in the expansion (the default is 0.1).

We consider two cases with `interaction.depth` equal to 3 and 1. The function `summary` returns a data frame where the first component is the variable name and the second is the computed relative influence, normalized to sum to 100, together with a graphical representation.

```
# boosting with d=3
set.seed(11)
hitter.boost3 <- gbm(log(Salary) ~ Hits+Years+RBI+PutOuts+Walks+Runs,
                     data=Hitters.compl, distribution = "gaussian",
                     n.trees = 5000, interaction.depth = 3)
summary(hitter.boost3) # plot with the regressor variable importance measures
```



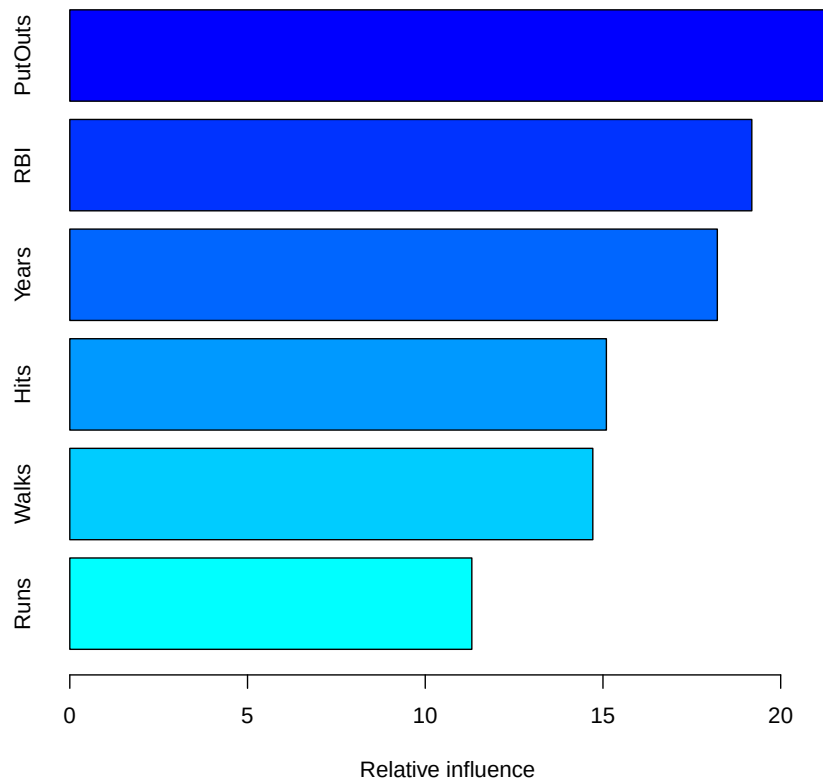
	var	rel.inf
Years	Years	27.64353
PutOuts	PutOuts	17.68494
Hits	Hits	15.60101
RBI	RBI	14.14262
Walks	Walks	14.02265
Runs	Runs	10.90525

```
# with d=1 (decision stamp)
set.seed(11)
hitter.boost1 <- gbm(log(Salary) ~ Hits+Years+RBI+PutOuts+Walks+Runs,
```

```

data=Hitters.compl, distribution = "gaussian",
n.trees = 5000, interaction.depth = 1)
summary(hitter.boost1) # plot with the regressor variable importance measures

```



var	rel.inf
PutOuts PutOuts	21.47519
RBI RBI	19.18737
Years Years	18.21704
Hits Hits	15.09574
Walks Walks	14.71426
Runs Runs	11.31039

Using the `crossval` function we obtain estimates for the test MSE based on a CV procedure with 10 folds. We have to define suitable functions to be considered for the bagging, random forest, boosting and optimal tree procedures.

```

### CV estimate of the testMSE
# CV testMSE bagging
predfun.bag <- function(train.x, train.y, test.x, test.y)

```

```

{
  fit.bag <- randomForest(train.x, train.y, mtry=6, importance=TRUE)
  yhat.bag <- predict(fit.bag,newdata=test.x)
  mean((yhat.bag-test.y)^2)
}

set.seed(22)
mse.cv.bag <- crossval(predfun.bag,as.matrix(training.regr), training.resp, B=1,
                      verbose = F)
mse.cv.bag$stat
[1] 0.2596993
mse.cv.bag$stat.se
[1] 0.05012257
# CV testMSE random forest with m=3
predfun.rf3 <- function(train.x, train.y, test.x, test.y){
  fit.rf3 <- randomForest(train.x, train.y, mtry=3, importance=TRUE)
  yhat.rf3 <- predict(fit.rf3,newdata=test.x)
  mean((yhat.rf3-test.y)^2)
}

set.seed(22)
mse.cv.rf3 <- crossval(predfun.rf3,as.matrix(training.regr), training.resp, B=1,
                      verbose = F)
mse.cv.rf3$stat
[1] 0.243276
mse.cv.rf3$stat.se
[1] 0.04666222
# CV testMSE random forest with m=4
predfun.rf4 <- function(train.x, train.y, test.x, test.y){
  fit.rf4 <- randomForest(train.x, train.y, mtry=4, importance=TRUE)
  yhat.rf4 <- predict(fit.rf4,newdata=test.x)
  mean((yhat.rf4-test.y)^2)
}

set.seed(22)
mse.cv.rf4 <- crossval(predfun.rf4,as.matrix(training.regr), training.resp, B=1,
                      verbose = F)
mse.cv.rf4$stat

```

```

[1] 0.2467812

mse.cv.rf4$stat.se

[1] 0.04809687

# CV testMSE boosting d=3
predfun.boost3 <- function(train.x, train.y, test.x, test.y){

  fit.boost <- gbm.fit(train.x, train.y, distribution = "gaussian",
                       n.trees = 5000, interaction.depth = 3, verbose = F)
  yhat.boost <- predict(fit.boost, newdata=test.x, n.trees=5000)
  mean((yhat.boost-test.y)^2)
}

set.seed(22)
mse.cv.boost3 <- crossval(predfun.boost3, as.matrix(training.regr), training.resp,
                          B=1, verbose = F)
mse.cv.boost3$stat

[1] 0.2838919

mse.cv.boost3$stat.se

[1] 0.06540744

# CV testMSE boosting d=1
predfun.boost1 <- function(train.x, train.y, test.x, test.y){

  fit.boost <- gbm.fit(train.x, train.y, distribution = "gaussian", n.trees = 5000,
                       interaction.depth = 1, verbose = F)
  yhat.boost <- predict(fit.boost, newdata=test.x, n.trees=5000)
  mean((yhat.boost-test.y)^2)
}

set.seed(22)
mse.cv.boost1 <- crossval(predfun.boost1, as.matrix(training.regr), training.resp,
                          B=1, verbose = F)
mse.cv.boost1$stat

[1] 0.2809465

mse.cv.boost1$stat.se

[1] 0.06419516

```

```

# CV testMSE for the optimal regression tree
set.seed(11)
hitter.tree1 <- rpart(log(Salary) ~ Hits+Years+RBI+PutOuts+Walks+Runs, data=Hitters,
                      method="anova", subset = complete.cases(Hitters),
                      control=list(cp=0.005))
cverror<-hitter.tree1$cptable[,4]
minxe <- which.min(cverror) # tree with the lowest cverror
hitter.tree1$cptable[minxe, 4]

[1] 0.4539

hitter.tree1$cptable[minxe, 5]

[1] 0.06828076

```

We now consider the `spam7` data set.

The function `J48` of the library `RWeka` generates unpruned or pruned C4.5 decision trees. This algorithm returns a (pruned) classification tree based on the cross-entropy node purity measure. Here we set the minimum number of instances per leaf (final node) equal to `M=10`, in the `control` argument. The graphical representation of the tree is given. The CV estimate, with 10 folds, of the confusion matrix is provided using the `evaluate_Weka_classifier` function.

```

##### trees
tree <- J48(yesno~crl.tot+dollar+bang+money+n000+make, data = spam7,
           control=Weka_control(M=10))
plot(tree)

```



```

bag <- randomForest(yesno~crl.tot+dollar+bang+money+n000+make, data = spam7, mtry=6)
# based on OBB data (asymptotically equivalent to leave one out CV)
cv.bag <- bag$confusion[1:2,1:2]
##### Random Forest
set.seed(11)
rf <- randomForest(yesno~crl.tot+dollar+bang+money+n000+make, data = spam7)
# based on OBB data (asymptotically equivalent to leave one out CV)
cv.rf <- rf$confusion[1:2,1:2]
ev.acc <- function(CM)
{
  acc <- (CM[1,1] + CM[2,2]) / sum(CM)
  tn <- CM[1,1] / (CM[1,1]+CM[1,2])
  tp <- CM[2,2] / (CM[2,1]+CM[2,2])
  return(c(acc,tn,tp))
}
tabe <- rbind(ev.acc(cv.tree), ev.acc(cv.boost),
              ev.acc(cv.bag), ev.acc(cv.rf))
colnames(tabe)<-c("tot. accuracy","true negative","true positive")
rownames(tabe)<- c("Trees","Boosting","Bagging","Random Forests")
tabe

```

	tot. accuracy	true negative	true positive
Trees	0.8698109	0.9361549	0.7677882
Boosting	0.8735058	0.9304161	0.7859901
Bagging	0.8758965	0.9167862	0.8130171
Random Forests	0.8837209	0.9505022	0.7810259